DFO AI Hub

AI/ML platform developed by the Office of the Chief Data Steward (OCDS) and Strategic Data Policy and Analytics (SDPA)

Authors: @Onak, David (DFO/MPO), @Yan, Hayden (DFO/MPO), and @Potiagalov, Alexander (DFO/MPO)

Contents

Contents 2
Section 1: Overview 4
1.1 Introduction 4
1.2 Coding framework overview 4
1.2.1 Frontend 4
1.2.2 Server 5
1.2.3 Backend 5
1.3 Management of keys 6
1.4 Running AI Hub Locally 6
1.4.1 Using Terminals 6
1.4.2 Using docker 8
1.5 Azure Deployment 9
1.5.1 Azure Resources Needed and Setup 10
1.5.2 Pushing a Deployment to Azure 16
Section 2: Frontend 19
2.1 Framework/Structure 19
2.2 Communicating with the Backend and Server 19
2.2.1 When Running in a Terminal Window 19
2.2.2 When Running with Docker 20
2.3 Authenticating Users into the Platform 20
2.3.1 Authentication Overview 20
2.3.2 Prerequisites 21
2.3.3 Configuration Structure 21
2.3.4 Application Integration 21
2.3.5 Login Behavior and Access Control Model 21
2.4 Language Toggling 22
2.5 Google Analytics 22
2.6 Adding a New Tool/Page 22

2.7 Current AI Hub Tools 26
2.7.1 Computer Vision 26
2.7.2 Large Language Models 29
2.7.3 Optical Character Recognition 32
2.7.4 Model Repo 32
2.7.5 AI Inventory Form 33
2.7.6 Statistical and ML Algorithms Guide 33
Section 3: Server 34
3.1 Framework/Structure 34
3.1.1 Purpose and Scope 34
3.1.2 Technology Stack and Architecture 34
3.2 Environment Configuration 34
3.3 Route Structure 35
3.4 Use Case: AI Intake Form Handling 35
3.4.1 Purpose 35
3.4.2 Architectural Role 35
3.5 Use Case: Document Retrieval and Editing (WebViewer Integration) 35
3.5.1 Purpose 35
3.5.2 Architectural Role 35
3.6 Relationship with Frontend 35
Section 4: Backend 36
4.1 Framework/structure 36
4.2 Environment Configuration and Azure Key Vault 36
4.3 Choma Storage in Volumes and File Share 37
4.4 Adding a New API Route 38
4.5 Supported APIs 39
4.5.1 HTTP Requests Handled Internally 39
4.5.2 External APIs Used 42

Section 1: Overview

1.1 Introduction

The DFO Office of the Chief Data Steward (OCDS) and Strategic Data Policy and Analytics (SDPA) AI Hub, a platform designed to explore the potential of artificial intelligence in fisheries and oceans research. Our initiatives harness the power of advanced data and A.I. technologies like machine learning, computer vision, and natural language processing to revolutionize how to support marine conservation and rebuilding efforts in the modern data and digital era.

These tools are prototypes designed to illustrate possible AI applications for DFO scientists, therefore, are strictly for educational purposes and are not to be used in any work processes. Please avoid uploading any sensitive or operational data if using this tool.

See GitHub repository for the codebase used, dfo-mpo/SDPA-AI-Portal. It also contains extensive documentation in markdown files. Before contributing any code to the AI Hub, read the CONTRIBUTE.md file.

1.2 Coding framework overview

The AI Hub is made up of 3 components:

- Frontend: Responsible for the interface users interact with and the authentication of users.
- Server: Handles requests from the frontend for forms and documents hosted on the AI Hub.
- Backend: Handles requests from the frontend for all AI, ML, and computer vision models including the model repository.

1.2.1 Frontend

The frontend is implemented using React JS (version 18.3.1), Node (version Node 22.12.0), and material UI (version 6.4.7) for many UI components. The code for this component can be found in the `./src` folder with `index.js` being the startup file/entry point. When using

docker, this components dockerfile attaches a nginx instance (see `nginx-app.conf` in the root folder) to serve as a reverse proxy for requests to the server and backend. The frontend is structured as so:

- components - files containing smaller, reusable components, including logic for authenticating users to the portal.
- context - files used to manage global states (i.e. user authenticated, or current language).
- hooks - contains reusable logic such as handling file upload for a button.
- layout - larger components that define the structure/layout of the AI Hub UI.
- pages - JS files that build each of the pages used in the AI Hub. Requests to the server are made directly from the pages that need to read or write to an Azure Blob Storage.
- services - logic for all requests made to the backend.
- styles - files containing CSS styling or overrides for material UI components.
- translations - files defining objects containing all English-French pairs for the AI Hub so it can support both languages.
- utils - helper and miscellaneous files used in the frontend.

See Section 2: Frontend for more details on this component.

1.2.2 Server

The server is implemented using express JS (version 5.1.0). The code can be found in the `./server` folder and runs on the startup of the frontend when used in terminal or as its own image when using docker. It is exposed to port 4000 but when docker is used, the port is only accessed via a reverse proxy. The server is structured as so:

- routes - contains logic for handling and responding to API requests.
- services - contains logic for establishing connections to cloud resources being used (Azure Blob Storage).

See Section 3: Server for more details on this component.

1.2.3 Backend

The backend is implemented using Python (version 3.10) with a FastAPI and Uvicorn framework. The code can be found in the `./backend/ai_ml_tools` folder and initiated with the main.py file. The backend is structured as so:

- routers - contains logic for handling and responding to API requests.
- models - contains Python classes used by the backend
- utils - contains Python helper functions for the routers
- core - contains any configuration files as needed
- data - contains cached responses for dev use

See Section 4: Backend for more details on this component.

1.3 Management of keys

Environment variables are used in both the backend and server components. This includes secrets and keys when running the scripts locally. However, when deploying on the cloud, the AI Hub supports retrieving all of its API keys from an Azure Key Vault resource. In this case, the '.env' files used are still important, but the values for any keys (as indicated by comments) can be left blank before dockerizing the Hub for deployment.

Details on how to setup the Azure Key Vault can be found in the 1.4.3 Deploying on Azure (using docker) section.

1.4 Running AI Hub Locally

1.4.1 Using Terminals

This method of running the AI Hub is recommended for development and is intended to be used on the 'local_dev' branch or any feature branch. The intention is that once a new feature is built and pulled into 'local_dev', it can be pulled into 'main' for testing dockerization and deployment.

Prerequisites

Ensure you have Python 3.10 and NPM installed on your system.

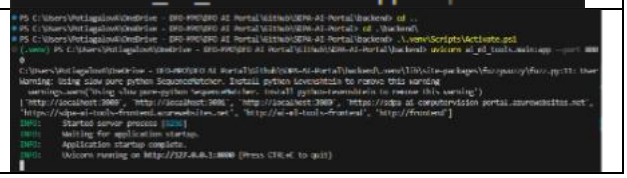
Running the backend will likely fail if you do not use the correct Python version. To support newer versions, all of the package versions outlined in the `requirements.txt` will need to be updated and tested.

Running the backend for the first time

Step Description	***Image***
Clone the git repository	dfo-mpo/SDPA-AI-Portal
Navigate to the backend directory	`cd .\backend\`
Create the virtual environment (This will create a .venv folder containing your isolated Python environment)	`python -m venv .venv`
Activate the virtual environment	`.\.venv\Scripts\Activate.ps1`
Install required dependencies	`pip install -r requirements.txt`
Start the backend server	`uvicorn ai_ml_tools.main:app --port 8000`

Verify success	`INFO: Started server process [8236]`
	`INFO: Waiting for application startup.`
	`INFO: Application startup complete.`
	`INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)`

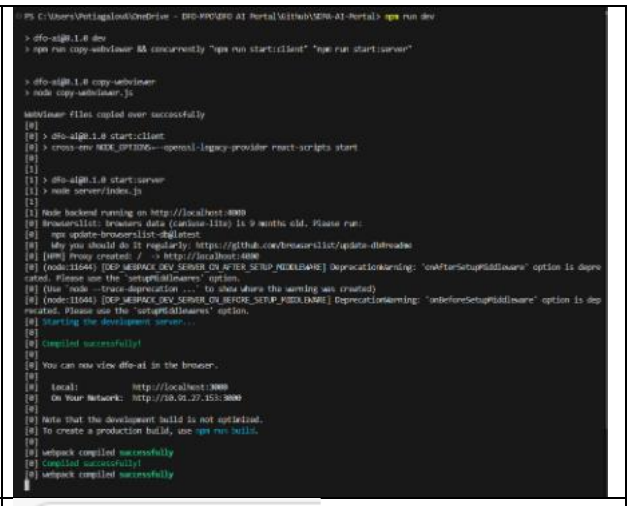
Running the backend again in the future

Step Description	***Image***
Navigate to the backend directory	`cd .\backend\`
Activate the virtual environment	`.\.venv\Scripts\Activate.ps1`
Start the backend server	`uvicorn ai_ml_tools.main:app --port 8000`
If successful, the whole setup should look something like this:	

Running the frontend and server

Step Description	***Image***

Open a new terminal at the project root level (important since both servers should be running in different terminals). Install all the required packages under package.json by running “npm install”	`PS C:\Users\PotiagalovA\OneDrive - DFO-MPO\DFO AI Portal\Github\SDPA-AI-Portal> npm install`
	
	`removed 1 package, and audited 1627 packages in 8s`
	
	`400 packages are looking for funding`
	`run `npm fund` for details`
	
	`19 vulnerabilities (3 low, 6 moderate, 9 high, 1 critical)`
	
	`To address issues that do not require attention, run:`
	`npm audit fix`
	
	`To address all issues (including breaking changes), run:`
	`npm audit fix --force`
	
	`Run `npm audit` for details.`
	`PS C:\Users\PotiagalovA\OneDrive - DFO-MPO\DFO AI Portal\Github\SDPA-AI-Portal>`

To start the frontend server, run the command “npm run dev” This will start BOTH the frontend and server.	
Visit this port URL: New tab	
Once the URL loads, you should be redirected to this landing page. (Note: It is important to have the backend running simultaneously as the frontend, otherwise features may not work as intended and only UI components will)	

1.4.2 Using docker

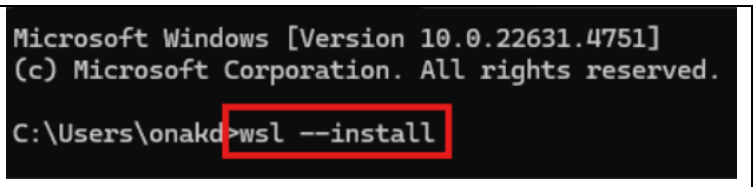
Running the AI Hub should be done when testing new features before deploying them to the cloud. This process will use local keys for APIs in the `.env` files.

The docker-compose.yml file will allow for a docker container be created using the frontend, server, and backend images. It reroutes the frontend exposure port to 3080 while rerouting the backend port to 8080.

Prerequisites

You will need to have docker on your system, if you don't have it already then follow the instructions below:

Step Description	***Image***
First you need to install the Linux subsystem for Windows, open a PowerShell or Command Prompt	

terminal using Admin Privileges and run the following command: wsl --install You may need to restart the computer after this step.	
Download and install Docker Desktop from the Docker Website (requires admin privileges) Windows | Docker Docs When prompted, ensure the **Use WSL 2 instead of Hyper-V** option on the Configuration page is selected	

Running the AI Hub

Step Description	***Image***
Build the local docker container using the command at the root of the project: docker-compose up --build If your docker version is 20.10 or newer, use this command instead: docker compose up --build	

You can now use the web app locally from, http://localhost:3080.

1.5 Azure Deployment

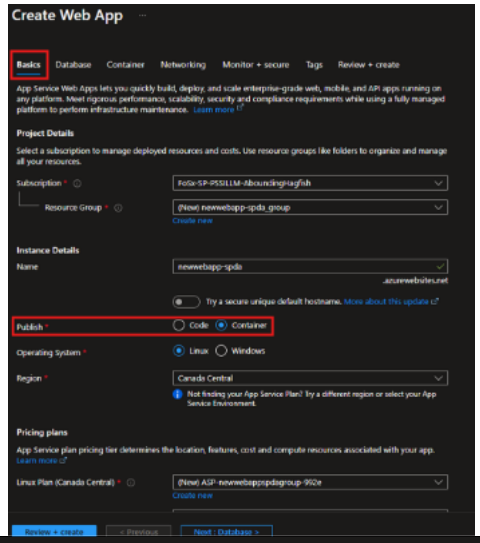
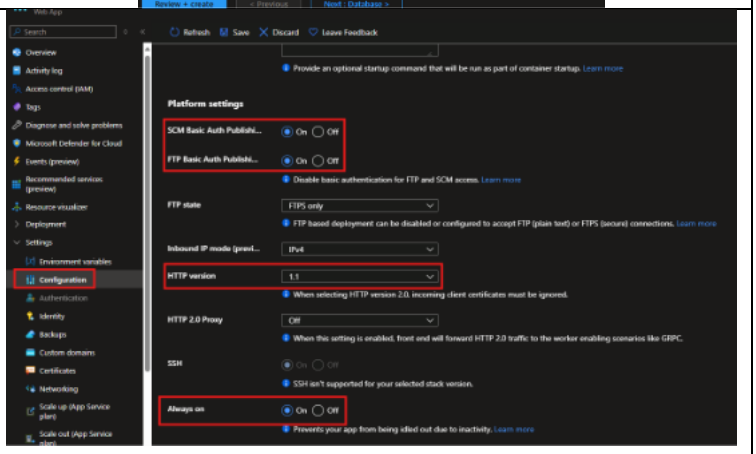
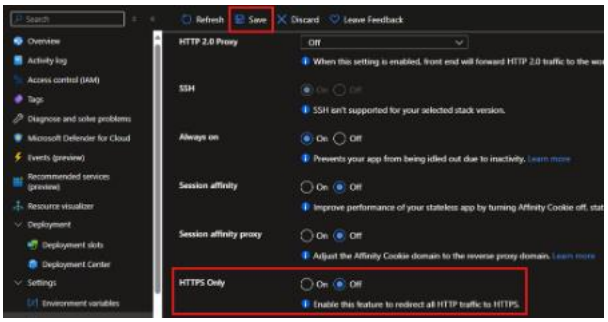
The deploy the AI Hub to the cloud the following resources are used:

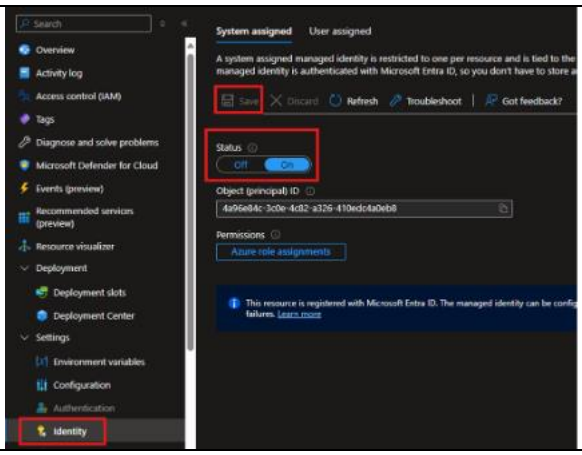
- Azure Web Apps: The deployment for the AI Hub
- Azure Container Registry: Hosting the docker images used by the Azure Web App
- Azure App Registration: Used for setting up authentication in the Azure Web App
- Azure Key Vault: Used to store keys for APIs used by the AI Hub

- Azure Storage Account: Blob storage used by the server in the AI Hub

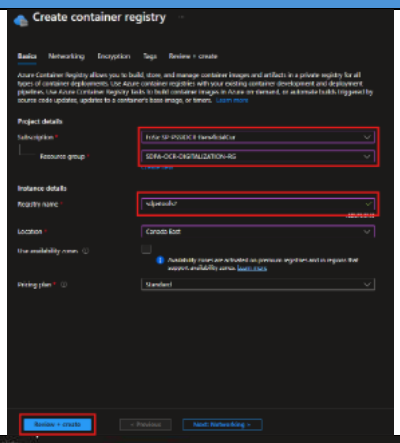
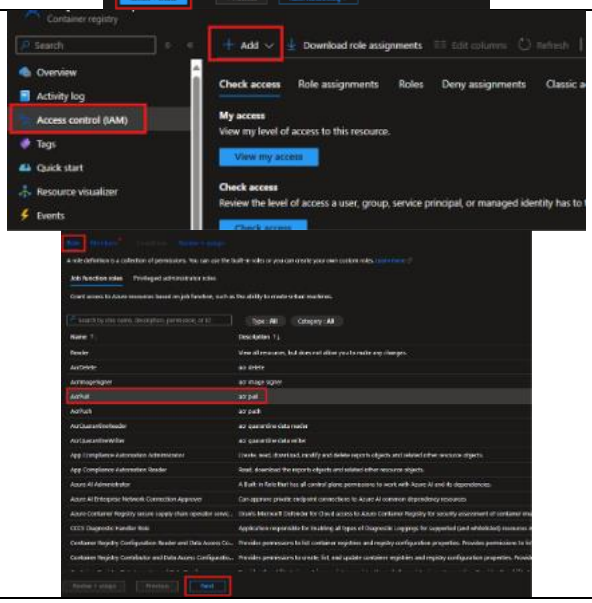
1.5.1 Azure Resources Needed and Setup

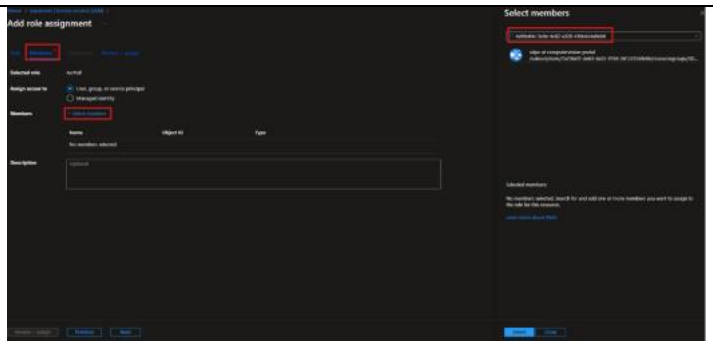
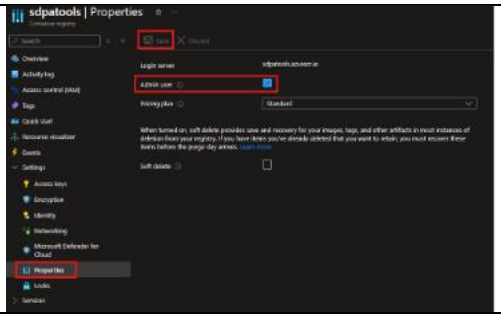
Setup an Azure Web App Resource

Step Description	***Image***
Create an Azure Web App resource. Make sure that the option for Publish is set to *Container*.	
Once the resource is created, open it and go into the *Configuration* page under settings and apply the following changes the click on 'save': • Set SCM Basic Auth Publishing Credentials to 'On' • Set FTP Basic Auth Publishing Credentials to 'On' • Set HTTPS Only to 'Off' (Only if any of your code makes external HTTP requests, the AI portal backend does to a VM with high GPU, make sure to understand the risks of non secure requests) • Set Always on to 'On' (Recomended) • Ensure HTTP version is set to 1.1, the proxy in frontend dockerfile is set up to use 1.1	 

Now go to the *Identity* page under the **System assigned** tab, set Status to 'On', press save, and copy the Object ID that is generated.	

Setup Container Registry Resource

Step Description	***Image***
Create an Azure Container Registry, using default configurations. Select the desired subscription, resource group, and give it a relevant resource name. Then click on 'Review + create'	
Once the resource is created, open it and go to the *Access control (IAM)* page. Then create a new role assignment for the Web App resource: • Click on the '+ Add' option to go to the role creation page. • Under 'Role' select *AcrPull* then click next. • Under 'Memebers click on *Select members* and under the Object ID copied from step 3 into the search bar. Select the web app resource that appears.	

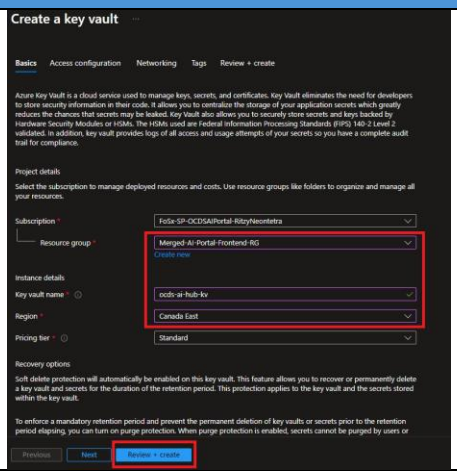
• Click Next and finish the creation of the role assignment.	
Now go to the *Properties* page and make sure the Admin user option is selected and click 'Save' to save the change.	

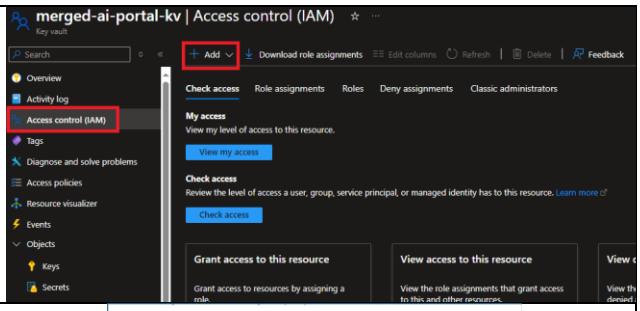
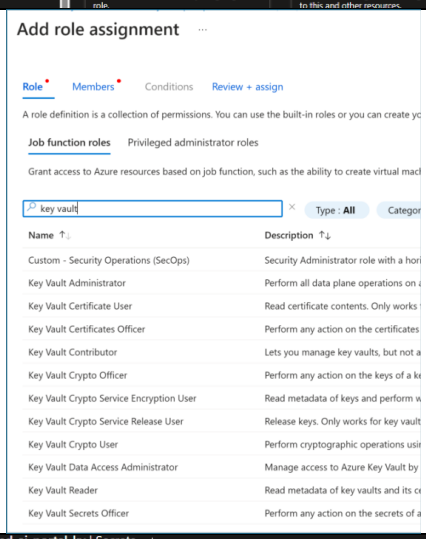
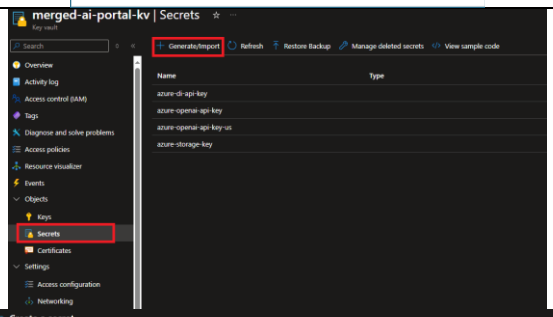
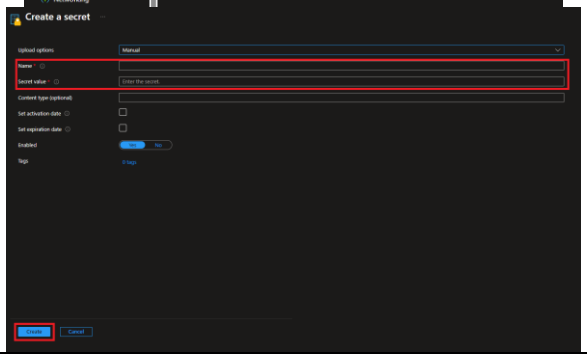
Setup App Registration

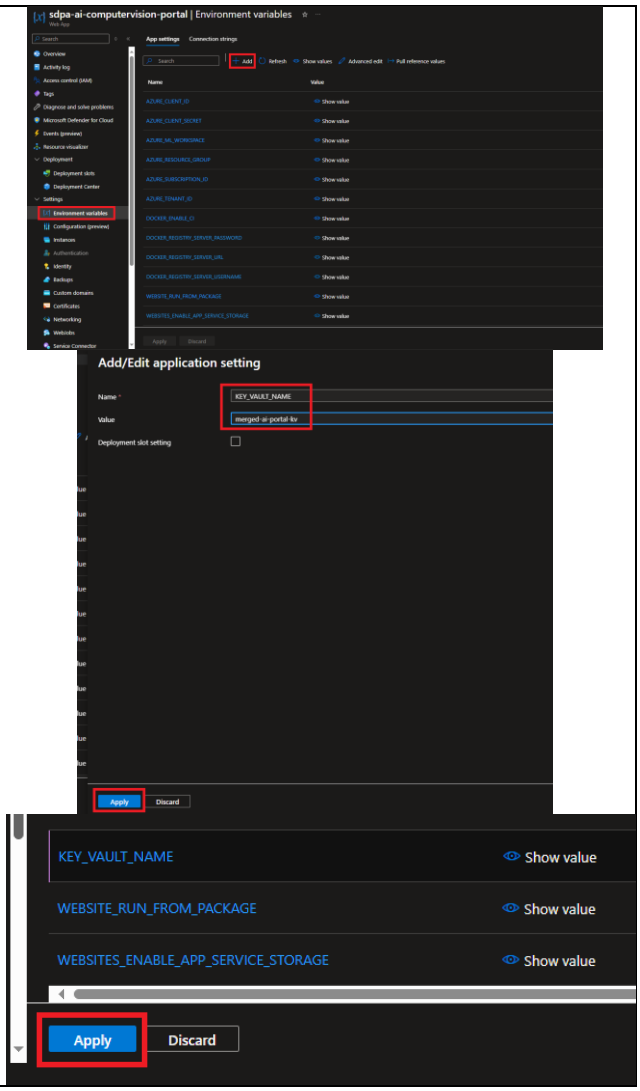
You will need to create an App Registration for the Azure Web App used and the users that will have the ability to authenticate when logging in. You will likely need to reach out to IT to create this, see this documentation for more details.

Get the application ID of the App Registration and the application ID of the service principle tied to the web application. They will be needed later.

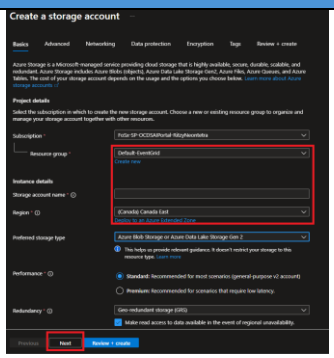
Setup Key Vault

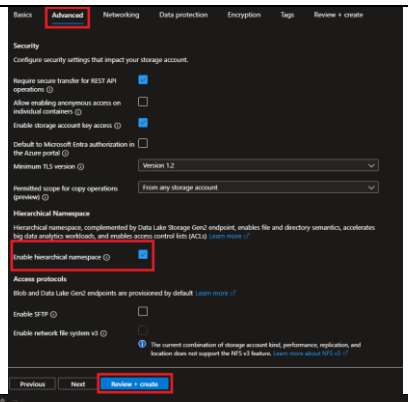
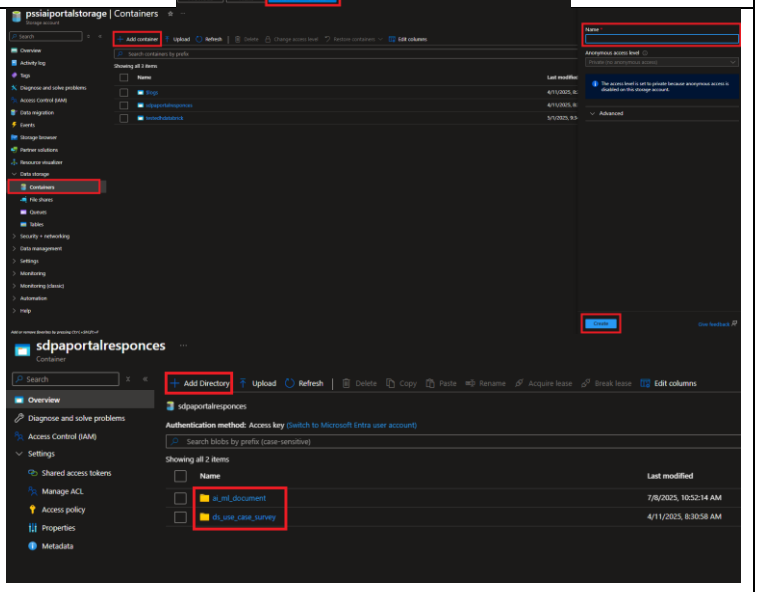
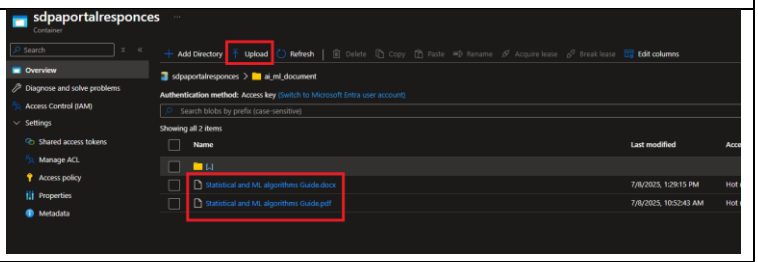
Step Description	***Image***
Create an Azure Key Vault, using default configurations. Select the desired subscription, resource group, region, and give it a relevant resource name. Then click on 'Review + create'	

Go to the resource once it is created and open the `Access Control (IAM)` page. The click on the `+Add` button.	
Create a new role granting the `Key Vault Contributor` role for the Object ID (as the member) for the Azure Web App **AND** another `Key Vault Contributor` role assignment for the application ID from the service principle tied to the Azure Web App.	
Go to the `Secrets` page and create secrets for each key as identified in the `./backend/ai_ml_tools/utils/azure_key_vault.py` and `server/services/azureBlobClient.js` files. The name of the keys must match as they are shown in these files as Azure Key Vault uses different naming conventions than what is used in `.env` files.	 

Return to the Azure Web App resource created earlier. Go to the Environment variables page. Click on Add to create a new environment variable: • Set the Name to KEY_VAULT_NAME • Set the Value to the name of your Azure Key Vault resource. • Click Apply to add the variable, then click Apply again to save the change.	

Setup Storage Account

Step Description	***Image***
Create an Azure Storage account resource. • Select the desired subscription, resource group, region, and give it a relevant resource name • (Recommended) Click next and on the `Advanced` window, select `Enable hierarchical`.	

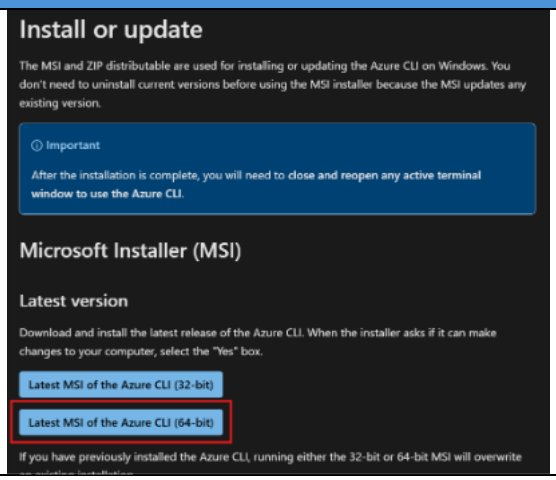
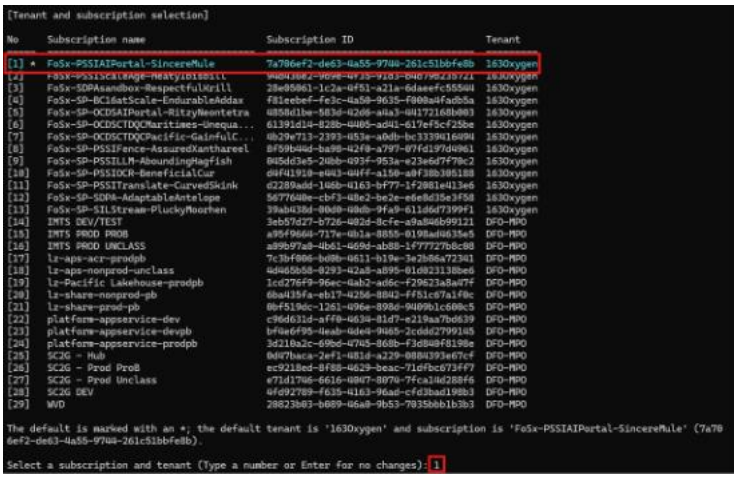
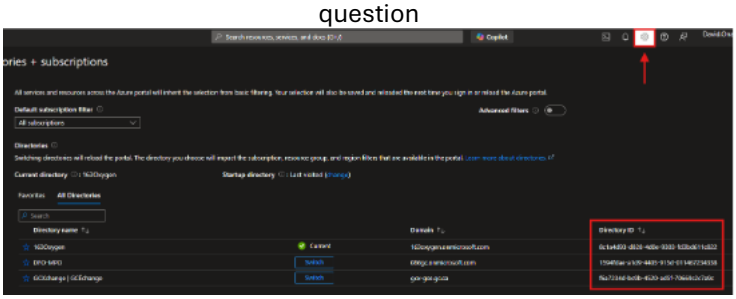
Then click on ‘Review + create’	
Once the resource is created, open it and go to the `Containers` page and add a new container (pick any name you want). In this container, create 2 folders: • **ds_use_case_survey**, this will be where survey responses are outputted. • **ai_ml_document**, you will need to upload a pdf and docx version of the **Statistical and ML Algorithms Guide** into this folder.	
Click on the **ai_ml_document** folder to open it and click on the upload button to upload the PDF and DOCX versions of the **Statistical and ML Algorithms Guide** document.	

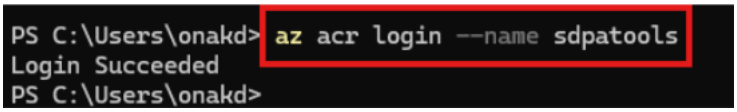
See additional setup needed in the Azure Storage Account if using the Web Scraper tool in section 4.3 Chroma Storage in Volumes and File Share.

1.5.2 Pushing a Deployment to Azure

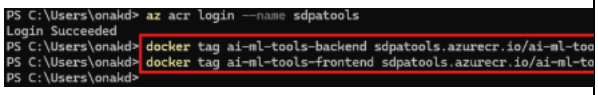
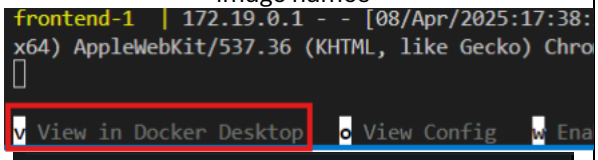
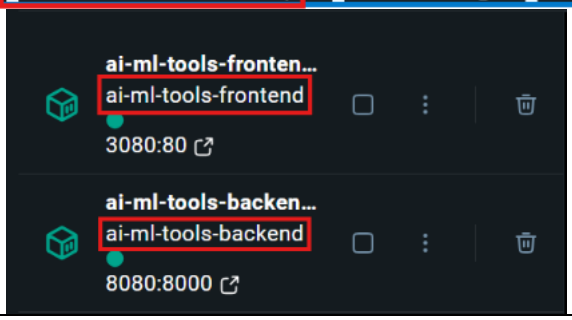
Before following these steps to update the Azure Web App deployment, make sure to have the AI Hub running locally first as outlined in the 1.4.2 Using Docker section.

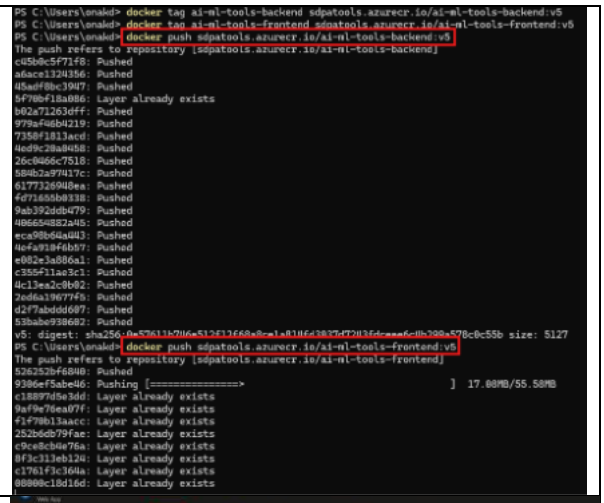
Setup Terminal Connection with Azure CLI

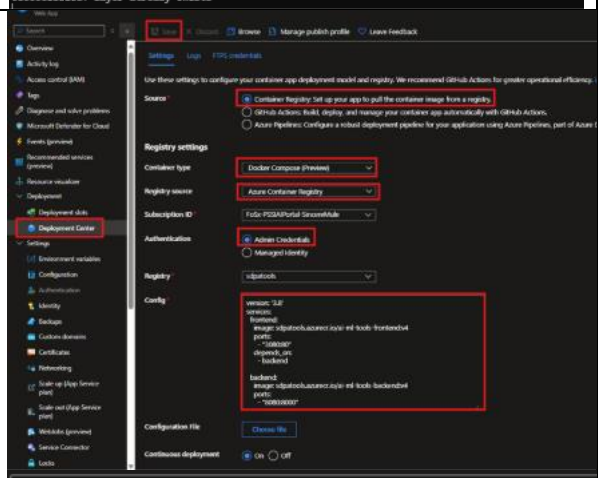
Step Description	***Image***
Download and install Azure CLI from Microsoft: Install the Azure CLI for Windows | Microsoft Learn After the download run the msi file. Note that you may need admin privileges to run the msi file.	
Open a terminal on your PC (such as Windows PowerShell) and log into the azure tenant you wish to use: Enter the following command: az login You will be shown all available subscriptions, select the one with your Azure Container registry. If the subscription you are looking for is in a tenant that does not appear, then you will need to login to that specific tenant: az login --tenant <tenantID> Note SSC 163Oxygen tenantID is 8c1a4d93-d828-4d0e-9303-fd3bd611c822. You can find it as Directory ID when viewing directories in Azure.	 In this example subscription number 1 has the ACR resource in question 

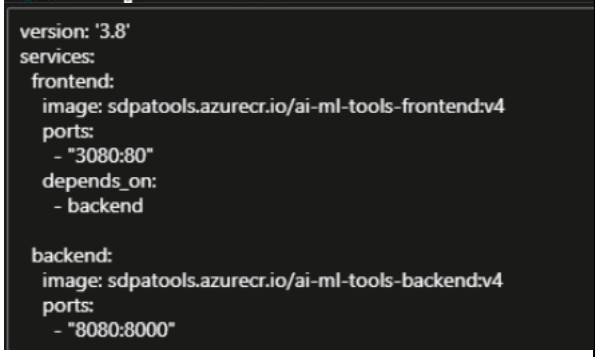
Log into your Azure Container Registry resource using the following command: az acr login --name <acr_resource> Where acr_resource is the name of the resource you want to push your docker images too.	

Create New Deployment on Azure Web App

Step Description	***Image***
Once you have built your local docker container go to back to whichever terminal you logged in with Azure CLI. You need to tag your images with the command: docker tag <your_image_name> <your_registry_name>.azurecr.io/<your_image_name>:<version> The version will be a number such as v1, v2, ect. Create a tag for each image you want to push to Azure.	 See images below on how you can get your docker image names  
Push your local images to the ACR with the following command: docker push <your_registry_name>.azurecr.io/<your_image_name>:<version> Run this command for each image you wish to push.	

<table>
<tr>
<td></td>
<td>


</td>
</tr>
<tr>
<td>
Go to your Azure Web App resource to the Deployment Center page then set up the container:
<ul>
<li>Make sure 'Source' is set to Container Registry.</li>
<li>Make sure 'Container type' is set to Docker Compose.</li>
<li>Set ‘Registry source’ to Azure Container Registry</li>
<li>Select the subscription containing your ACR</li>
<li>'Authentication' Needs to be set to Admin Credentials</li>
<li>For 'Config' enter the following compose file, see example in the images</li>
<li>(Optional) Set 'Continuous deployment' to 'On' if you want the website to update if push a new image versions.</li>
</ul>
Press Save, after your web app refreshes it will be running of the frontend and backend containers.
</td>
<td>




Where ai-ml-tools-frontend and ai-ml-tools-backend were names of the local images pushed to ACR
</td>
</tr>
</table>

Section 2: Frontend

2.1 Framework/Structure

The frontend component is a React JS project located under `./src`. This component creates an interface where users interact with the AI Hub and its various tools.

The frontend component does not have its own direct backend and processing any of the hosted tools is done by making requests to the server and backend components.

Using a terminal shell, the frontend can be started by simply using the `npm run dev` command (which also starts up the server component). When using Docker, it is deployed as its own container. In this case, it is packaged with a nginx that servers as a reverse proxy.

All dependencies used in this component are defined in the `package.json` and `package-lock.json` files.

2.2 Communicating with the Backend and Server

All requests made to the backend component are handled in the `/services/apiService.js` file using helper functions called by various tool pages needing the API.

All requests to the server component directly in the `/pages/SurveyForm/SurveyForm.jsx` and `pages/DocxEditor.js` files.

The file `src/services/apiService.js` is included in the gitignore so when the `local_dev` branch is pulled into `main` the routing method is not overwritten. If new APIs are added, they will need to be manually pasted in.

2.2.1 When Running in a Terminal Window

The configuration in the frontend files to communicate to the server and backend using the terminal is only setup in the `local_dev` and feature branches. This will not work in the `main` branch.

Due to the defined paths in `./server/index.js`, when the server is run with the frontend in a terminal, all server requests made use `/api` as the API endpoint.

The API endpoint for all backend requests use the port number that the backend FastAPI is running on, such as `localhost:8000` if running on port 8000.

2.2.2 When Running with Docker

The configuration in the frontend files to communicate to the server and backend using the Docker is only setup in the `main` branch. This will not work in other branches.

When hosting the AI Hub in the cloud, an nginx reverse proxy is required to securely manage communication between the frontend, server, and backend containers. In a Dockerized and cloud-hosted environment, each component runs as its own service on an internal network. Directly exposing these services would require multiple public ports and could unintentionally expose non-secure HTTP or WebSocket (WS) endpoints to users.

To avoid this, the frontend container includes an nginx instance that serves as a single secure entry point for all browser traffic. The proxy accepts HTTPS and secure WebSocket (WSS) requests from the client (rather than HTTP and WS in the `local_dev` branch) and forwards them internally as HTTP and WS traffic to the appropriate service. This ensures secure external communication while keeping internal service traffic private.

Due to the reverse proxy's rerouting of request paths, any requests to the server need `/server` as the endpoint while the any to the backend uses `/api`.

The nginx-app.conf file is used to build the reverse proxy used by the frontend. Some of the configurations in this file include the line `client_max_body_size 10M;` which limits the size of all file uploads on the AI Hub to 10 megabytes.

2.3 Authenticating Users into the Platform

2.3.1 Authentication Overview

User authentication within the AI Hub frontend is implemented using Microsoft Entra ID through the Microsoft Authentication Library (MSAL) for Reach.

The implementation follows Microsoft's official guidance for React-based Page Applications (SPA):

- Get started with MSAL React - Microsoft Authentication Library for JavaScript | Microsoft Learn
- Tutorial: Prepare a React single-page application for authentication - Microsoft identity platform | Microsoft Learn

The application uses the `@azure/msal-react` and `@azure/msal-browser` packages to handle authentication requests.

2.3.2 Prerequisites

Authentication requires a registered application in Microsoft Entra ID configured as

- Application type: Single Page Application (SPA)
- Platform configuration: Redirect URI configured for the deployed URL

The Client ID and Tenant ID are configured in the frontend authentication configuration file.

2.3.3 Configuration Structure

MSAL configuration is defined in `src\components\auth\authConfig.js`.

This configuration follows the structure provided in the MSAL React documentation, with environment specific credentials. The configuration initializes an MSAL instance used across the application.

2.3.4 Application Integration

MSAL is integrated at the root of the React application. In `./src/App.js`, the application is wrapped in:

- MsalProvider: provides authentication context across the component
- AuthContext: custom lightweight wrapper used to simplify authentication state checks within UI components

This design allows authentication state to be accessed globally without tightly coupling components to MSAL specific hooks.

Authentication state is exposed via `AuthContext`, and UI components consume authentication state via `useAuth()`.

2.3.5 Login Behavior and Access Control Model

The AI Hub does not enforce mandatory login on initial page load. Instead, it fllows a conditional access visibility model:

- Users may browse publicly accessible content without authentication.
- Certain components and features remain restricted until the user is authenticated.
- When unauthenticated, a login button is displayed in the header; if authenticated, the login button is replaced by logout functionality.

2.4 Language Toggling

Do the unreliability of automatic translation tools/libraries especially in a DFO context; a custom built in tool is used for toggling language between English and French. Each piece of English text needs to have a French translation (acquired manually) in a structured object in the `translations/` folder. Each page in the AI Hub then needs to use these defined objects in place of any text, so what is generated depends on the current language selected.

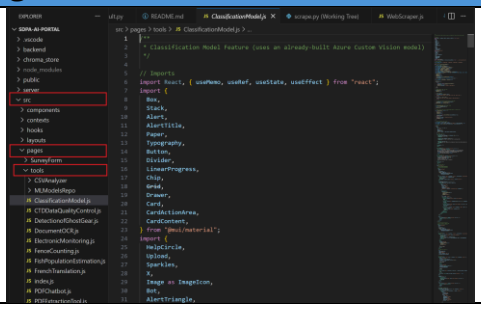
When the language toggle button is used, the `contexts/LanguageContext.js` file is used by the toggle and all AI Hub pages to keep track of what the current language is.

2.5 Google Analytics

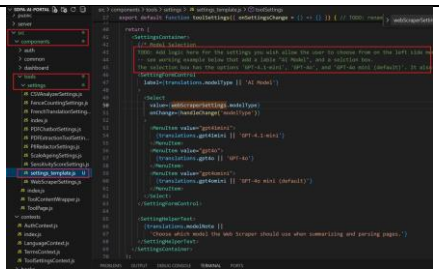
Google Analytics has been setup to track various events and pages in the AI Hub to better understand user behavior. The `react-ga4` library is used in the file `src/utils/analytics.js` to set up helper functions to track page views and specific events. It is then called in `src/App.js` to initialize the Google Analytics connection and page tracking. Any page in the AI Hub can then track specific events by importing `trackEvent()` from `analytics.js` and calling it during an event (such as a button click or API request failure). The name of the tool, description of the event, and label of the event type should always be passed to the `trackEvent()` function.

For documentation on setting up and tracking events in a Google Analytics workspace see the document [How to Track Web App Traffic Using Google Analytics 4.docx](#).

2.6 Adding a New Tool/Page

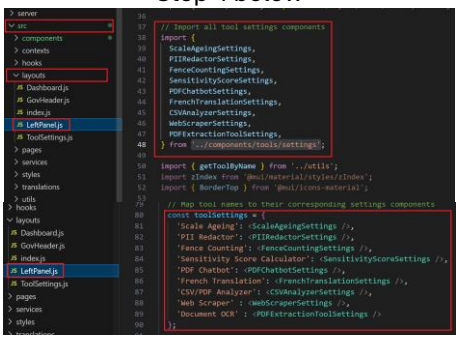
Step Description	***Image***
Go to the `pages/tools/` folder and create a new JS file for your tool. Add the logic this tools UI.	
If your tool has parameters that require user input, you can set those up in the left side menu. To do so, follow these steps:	Step 1 + 2 below

1. Go to the `src/components/tools/settings` folder and copy the `settings_template.py` file then rename it with your tool name.
2. Address all the `TODO` comment statements in the template and add in your settings logic.
3. Within the same folder, go to the `index.js` file and add your new settings component as an export.
4. Go to the file `src/layouts/LeftPanel.js` and add your settings component to both the `../components/tools/settings` import and to the `toolSettings` dictionary, add your tool name as the key.
5. Go to the file `src/contexts/ToolSettingsContext.js`, create a new React `useState` for the left panel parameter default states. Then add a function to update the React state with new parameter values. Include both in the `value` object at the bottom of the file.
6. (Optional) To prevent issues with passing the parameters to the backend, cleaning/processing can be done to ensure compatibility. Go to the file `src/utils/settingsAdapter.js` create a new export function for your tool left panel parameters and add it to the adapters dictionary at the bottom of the file.

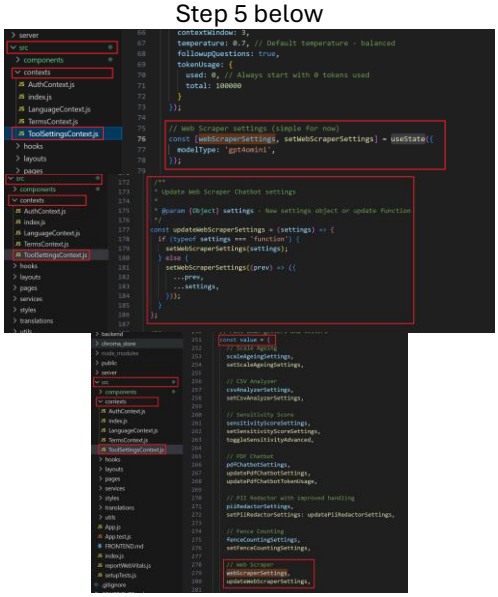


Step 3 below

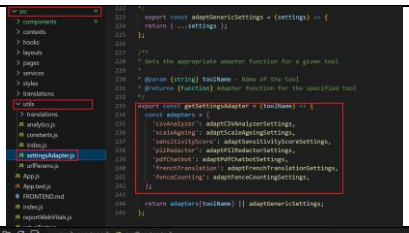
Step 4 below

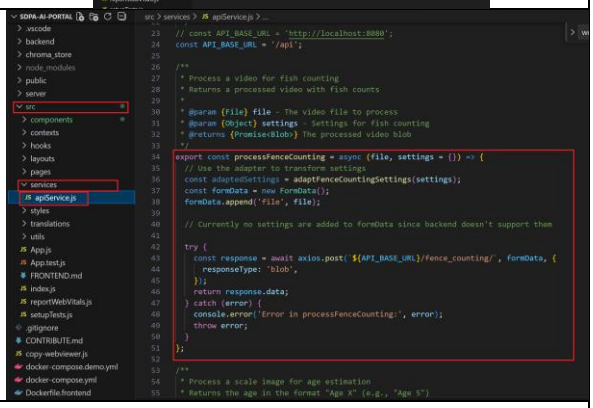
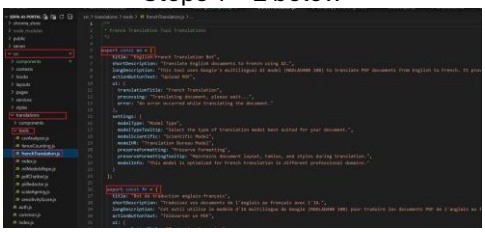
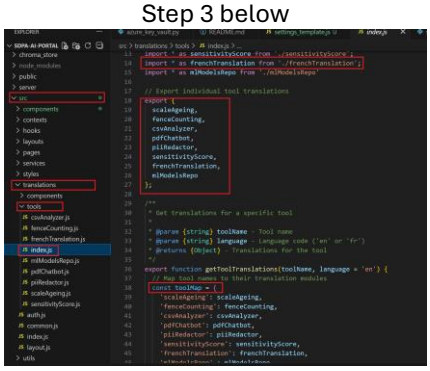
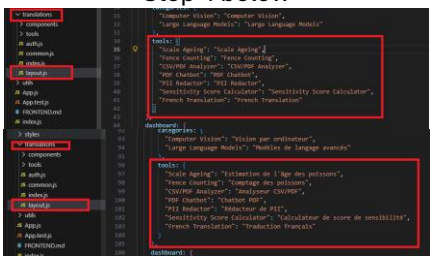


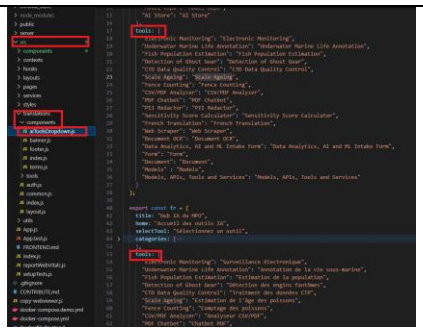
Step 5 below



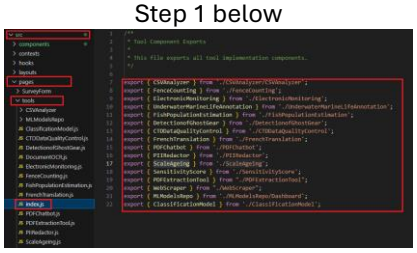
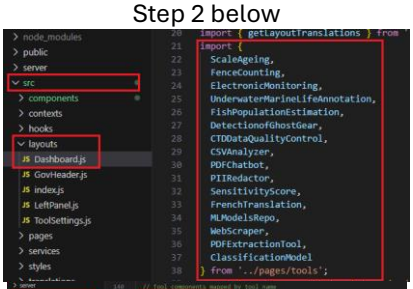
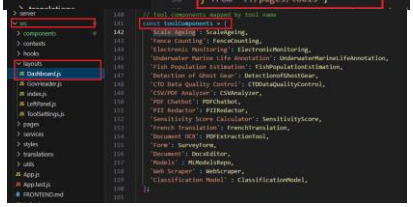
Step 6 below

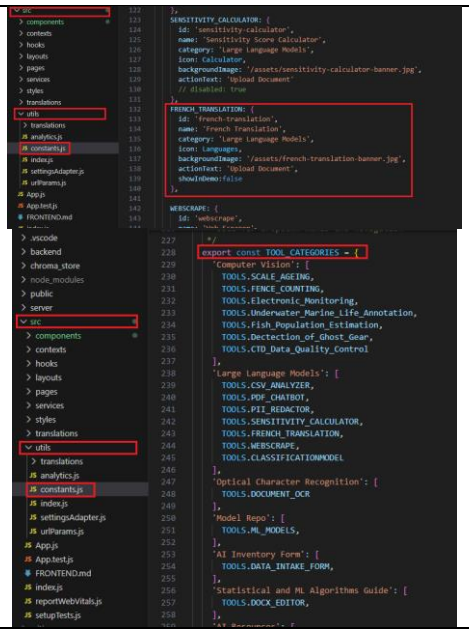
<table>
<tr><td></td><td>  </td></tr>
<tr><td>For any calls/requests made to the backend call one of the functions defined in `services/apiService.js`.

If you need to add a new API route for the backend:
1. Follow section [4.4 Adding a New API Route].
2. Create a new function in `services/apiService.js` that makes this new API request.</td><td>  </td></tr>
<tr><td>Add in the language/translation logic:
1. Create a new JS file for your tool in the `translations/tools/` folder.
2. In this new file, export 2 dictionaries, one called `en` for all text used in the tool's UI page and `fr` for all the translated text. Both dictionaries must share the same keys.
3. In the same folder, go to the `index.js` file. Import the translation page, then add it to both the export and the `toolMap` dictionary.
4. Add in the name of your tool in English and French in the list of tools found in `translations/layout.js` and `translations/components/aiToolsDropdown.js` files.
5. In the `utils/translations/toolTranslations.js` file, add a mapping for the new translation page for the tool. The name used should match the filename and will be key used in the UI tool page for accessing its English/French text.
6. In the new tool UI file, import `useLanguage` from the contexts folder and `getToolTranslations` from utils. You can then retreive the language dictionaries as so:
const { language } = useLanguage();</td><td>Steps 1 + 2 below
 
Step 3 below
 
Step 4 below
  </td></tr>
</table>

<table>
<tr>
<td>const toolData = getToolTranslations("toolName, language);</td>
<td>

Step 5 below

Step 6 below

</td>
</tr>
<tr>
<td>Add tool to menu options so users can see the tool by following these steps:
1. Go to the `src/page/tools/index.js` file and add your new tool UI page as an export.
2. Go to the `src/layouts/Dashboard.js` file, import your tool's UI page, and add it to the `toolComponents` dictionary.
3. Go to the file `src/utils/constants.js` define a new object for your tool, then add your new object to the `TOOL_CATEGORIES` export under the most relevant section (for example if the tool uses OpenAI put it under Large Language Models)</td>
<td>
Step 1 below

Step 2 below


Step 3 below
</td>
</tr>
</table>



2.7 Current AI Hub Tools

This section covers all of the tools currently deployed on the AI Hub. More technical details on how some of these tools are implemented in the backend are found in section 4.5 Supported APIs.

The source code for all the UI pages for all these tools are found in the `./src/pages/tools/` folder except for the AI Inventory Form located in `./src/pages/SurvayForm/` and the Statistical and ML Algorithms Guide UI found in `./src/pages/DocxEditor.js`.

2.7.1 Computer Vision

Scale Ageing

This tool uses a computer vision model trained on annotated salmon scale images to estimate fish age with high accuracy. It automates ageing for fisheries research and management.

The tool originates from a pilot project to help reduce the manual workload currently born by subject matter experts (SMEs) for the DFO Science Branch's Sclerochronology Lab (Fish Ageing Lab). During this pilot, a YOLOv9 model was developed that achieved high accuracy in detecting scale features (clipping, center points, and fragments). The tool also provides transparent explanations by identifying and displaying winter ring patterns corresponding to age determination.

The model itself is hosted on premise which is accessed via the backend component.

Fence Counting

This tool uses computer vision to analyze each frame of river monitoring videos, detecting and classifying salmon as they pass through counting fences. The AI model helps automate species identification and improve accuracy in population tracking.

This tool originates from a pilot project to help DFO's Science Branch's Stock Assesment reduce time spent on manually review of underwater footage to count and identify migrating salmon and therefore minimize delays in data availability.

During the pilot, a YOLOv11 detection model was successfully trained and integrated with BOT-SORT tracking algorithms to identify and count five salmon species: Pink, Chum, Chinook, Sockeye, and Coho. The model achieved 87% precision and 88% recall, demonstrating near-expert-level performance. When deployed, this system tracks individual fish throughout video segments and generates automated counts, eliminating the need for frame-by-frame manual review.

Due to the file sizes of videos and the processing time of the footage, the AI Hub only has sample videos that have been processed with the YOLOv11 model in advance.

Electronic Monitoring

This tool is a proof-of-concept demo for DFO's Automated Electronic Monitoring. The project aims to modernize the currently labour-intensive process of manually reviewing video footage of selected sets of tows. The underlying fish counting model was developed in collaboration with the Pacific Groundfish EM Program and various industry partners. The model is capable of identifying and counting in real-time commonly harvested fish species in imagery collected from electronic monitoring equipment onboard commercial fishing vessels.

The application is hosted on a different web application used by the OCDS team using Gradio. The version on the AI Hub is created by using an iframe rendering the Gradio page. As a result, it has its own English/French toggle that is independent of the AI Hub's toggle.

Underwater Marine Life Annotation

Proof-of-concept demo for DFO Underwater Benthic Marine species identification. This project proposed by Quebec Region's science team aims to automate the review of underwater imagery and video footage collected for biodiversity surveys. The model

developed is currently capable of identifying 21 benthic marine life categories drawn from underwater remotely operated vehicles. The project has also since garnered support from the science teams of other regions including Pacific Region.

The application is hosted on a different web application used by the OCDS team using Gradio. The version on the AI Hub is created by using an iframe rendering the Gradio page. As a result, it has its own English/French toggle that is independent of the AI Hub's toggle.

Fish Population Estimation

This tool is a proof-of-concept demo for an AI-based tool for DFO's underwater fish population estimation tasks. This project explores the potential of a tool that automates detection and estimation of the number of fish in underwater shoals and schools in imagery collected from camera equipment during underwater surveys. The model was developed as a by-product of the EM project, but it has since garnered interest from various programs like the Pacific Salmon Strategy Initiative where we are investigating the potential to leverage this model to automate salmon identification and counting.

The application is hosted on a different web application used by the OCDS team using Gradio. The version on the AI Hub is created by using an iframe rendering the Gradio page. As a result, it has its own English/French toggle that is independent of the AI Hub's toggle.

Detection of Ghost Gear

Proof-of-concept demo for DFO's Side-scan Sonar Image Ghost Gear Detector. This project was undertaken in partnership with the Ghost Gear Program and aims to automate the process of reviewing side-scan sonar imagery. The system developed leverages an AI-based computer vision model trained on data provided by CSR GeoSurveys Ltd. to identify in real-time Ghost Gear (Abandonned Lobster Traps) from collected side-scan sonar imagery.

The application is hosted on a different web application used by the OCDS team using Gradio. The version on the AI Hub is created by using an iframe rendering the Gradio page. As a result, it has its own English/French toggle that is independent of the AI Hub's toggle.

CTD Data Quality Control

Proof-of-concept demo for DFO Pacific Region CTD (Conductivity-Temperature-Depth) Data Quality Control model. CTD profiles are depth-wise series' of sensor measures for

oceanographic data taken at fixed locations. Challenging ocean conditions and sensor faults can lead to poor quality data that must be manually identified and removed by oceanographers. This project was developed in collaboration with the Pacific Region Ocean Sciences Division and aims to accelerate the CTD quality control process by flagging bad data to assist oceanographers in more rapidly identifying and removing the bad data. Through experimental results, the model achieves 92.6% global accuracy in identifying bad data. The project has also since gained interest from the oceanography teams of other regions including Maritime Region.

The application is hosted on a different web application used by the OCDS team using Gradio. The version on the AI Hub is created by using an iframe rendering the Gradio page. As a result, it has its own English/French toggle that is independent of the AI Hub's toggle.

Classification Model

This tool sends your image to the already-built Azure Custom Vision models and returns a predicted label + confidence. All models are accessed via the backend component of the AI Hub.

2.7.2 Large Language Models

CSV/PDF Analyzer

This tool enables structured document analysis by processing CSV-based prompts against PDF files. Users can define specific questions or extraction tasks in a CSV file, and the tool will analyze the uploaded document accordingly.

This tool originates from pilot projects aiming to use OCR and OpenAI to extract and summarize data from documents. A prebuilt OCR extraction model from Azure Document Intelligence is used to process the uploaded document, and then Azure OpenAI is used on the prompts defined in the CSV file to summarize relevant data with source references. Prompt engineering techniques (question formation and structuring) developed throughout the pilot are used on all prompts in the provided CSV file.

PDF Chatbot

This tool uses OpenAI's language model to answer questions about uploaded documents. It provides direct responses with sourced references, making document exploration faster and more efficient.

Like the CSV/PDF Analyzer, this tool originates from pilot projects using OCR and OpenAI to extract and summarize data from documents; however, it does one prompt at a time and uses retrieval-augmented generation (RAG) using chromadb. RAG allows only relevant chunks of the uploaded document to be passed to OpenAI for a given prompt, reducing processing time and cost.

PII Redactor

This tool leverages Microsoft Presidio to detect and redact Personally Identifiable Information (PII) such as names, addresses, and phone numbers in PDF documents. It helps enhance data privacy by automatically censoring sensitive content.

OCR is used to extract the text from the uploaded PDF and after Presidio is used, the uploaded PDF is modified with black boxes placed over all text determined to be sensitive content. This tool works well in some cases but needs to be future explored and developed to have useful applications in DFO.

Sensitivity Score Calculator

This tool uses Microsoft Presidio to analyze documents and determine their sensitivity score based on the presence of Personally Identifiable Information (PII). The higher the score, the more likely a document contains sensitive information.

OCR is used to extract the text from the uploaded PDF and after Presidio is used, a score is calculated based on the type of sensitive data detected and the number of detections. This tool works well in some cases but needs to be future explored and developed to have useful applications in DFO.

French Translation

This tool uses Google's multilingual AI model (MADLAD400 10B) to translate PDF documents from English to French. It provides fast and efficient translations for various types of content while maintaining context and readability.

This tool originates from a pilot project aimed to create a LLM that can translate English to French at the level of the French Translation Bureau. Training was done by collecting English-French text pairs from translated documents and fine-tuning an existing LLM with the training data. Although the pilot has been on hold due to lack of training documents

and capacity, the latest iteration was proven to provide more accurate translations in a DFO context compared to other publicly available LLM tools.

The AI model for this tool is hosted on premise and accessed via the backend.

Web Scraper

This tool allows users to input a website URL and automatically scrape its contents for structured data extraction. Once scraped, users can ask questions about the page using OpenAI-powered analysis. Ideal for quick insights, research, or prototyping, this scraper simplifies the process of turning raw web content into actionable answers.

At a high level, the solution works in two ways:

1) Scrape once, reuse many times

   A user pastes in a public website or intranet URL they care about. The tool automatically visits that site and its key subpages, scrapes the HTML content and reads commonly attached files and splits that content into smaller chunks. Each chunk is converted into an embedding and stored, along with its metadata, in a vector database. This makes the content searchable and reusable. Once a site has been processed, it appears as a “preset card” in the left-hand panel of the Web Scraper page, showing basic details like the site name and last scraped time.

2) Chat instead of text-hunting

   Once a site has been scraped, anyone with access to the AI Portal can select its preset and start chatting with it. Users can ask questions, and the chatbot responds based only on the text that was collected from that site. Behind the scenes, the chatbot looks up the most relevant chunks from the vector database for that site and uses them to generate an answer based only on the prompt. The chat history and the scraped content for each site can be downloaded as simple text files, making it easy to keep a record of what was asked, answered, and scraped.

From a user’s perspective, this turns a complex website into a single, interactive experience. Over time, a set of commonly used websites can be scraped once and reused by many scientists, analysts, and teams.

For more detailed and technical documentation specifically on this tool, see the WebScraper Chatbot Documentation.

2.7.3 Optical Character Recognition

Document OCR

This tool allows users to upload a PDF and automatically scrape its contents for structured data extraction. Once scraped, users can ask questions about the PDF using OpenAI-powered analysis. Ideal for quick insights, research, or prototyping, this tool simplifies the process of turning raw PDF content into actionable answers.

This tool uses Azure Document Intelligence and Azure OpenAI via the backend component.

2.7.4 Model Repo

Models

A unified repository where users can upload, explore, and manage machine learning models. It supports versioning and model cards with key metadata to help teams quickly integrate models into their workflows. Users can browse existing models from OCDS and SDPA or contribute their own.

The solution introduces a model repository within the AI Hub, designed specifically to showcase the models that scientists have already built and trained. Rather than performing any machine learning tasks itself, the repository acts as a window into what already exists in Azure Machine Learning workspace which includes model versions, metadata, READMEs, tags, and artifacts stored within the MLflow registry server and Blob Storage.

Users continue to upload, version, and manage their models entirely through the AML workspace, while the repository simply exposes this information through the AI Hub. The backends' endpoints communicate directly with AML to retrieve lists of models, fetch metadata, download artifacts, and extract READMEs. The frontend then presents this information in a consistent, searchable, and user-friendly catalogue. The result is a centralized hub that makes internal models easy to discover and reuse while keeping all model lifecycle operations inside AML.

For more detailed and technical documentation specifically on this tool, see the ML Model Repo Documentation.

2.7.5 AI Inventory Form

Form

This form is meant to be use as guide to support business users in articulating and imagining the potential of using Data, Artificial Intelligence, and Machine Learning to improve productivity, efficiencies, and generate value for program and service delivery, operations, and other business processes. It is meant to help users determine the value and potential of new data innovation from a value proposition, scalability, and sustainability lens rather than a technical implementation perspective. Complete responses will help Data and AI Scientist to determine how feasible a solution could be.

Responses are handled by the server component which outputs into the Azure Storage Account connected to the AI Hub under the `ds_use_case_survey` folder.

2.7.6 Statistical and ML Algorithms Guide

Document

This document will describe different categories of machine learning algorithms and their intended purposes, specifically supervised learning followed by unsupervised learning. Individual algorithms will be explained, and a template will be provided for each algorithm that can be used out-of-box.

This document is strictly scoped to the selection and training of ML models, which are a subset of tasks within the broader domains of ML Ops and AI Governance. There are many other tasks and responsibilities that constitute effective and responsible development, deployment and usage of ML models. The breadth of information needed to fully cover these domains requires a suite of policy, guidance, and educational materials. This document can be seen as one component of this broader suite that is currently being assembled within DFO.

The library `@pdftron/webviewer` is used to generate the PDF view which allows users who are authenticated into the AI Hub to edit and annotate the document guide which is saved onto the file stored in Azure Blob Storage via the server component.

Section 3: Server

3.1 Framework/Structure

3.1.1 Purpose and Scope

The server component is a Node.js/Express service located under `./server`. This component is distinct from the AI Hub's primary "backend" (FastAPI service). It exists specifically to support:

1. AI Intake Form: Survey submission handling
2. Document retrieval and editing (WebViewer package integration)

The server component does not provide AI model functionality or VM communication. Its responsibility is limited to handling blob-based data storage required by frontend components.

The server is deployed as a separate Docker container. It is distinct from the FastAPI backend container.

3.1.2 Technology Stack and Architecture

The server is implemented mainly using Node.js, Express.js and `@azure/storage-blob`. It exposes REST-style API endpoints under `/api/*`.

The server acts as an API layer between "Frontend (React) -> Express Server -> Azure Blob Storage". All persistent data handled by this server is stored in Azure Blob Storage. No database is used.

3.2 Environment Configuration

The server uses environment variables stored in `server\.env`. The required environment variables are used to construct a connecting string dynamically inside `server\services\azureBlobClient.js`. The Azure Blob client is instantiated per request using the Azure SDK.

3.3 Route Structure

The server organizes functionality into modular route files under `server\routes`. Each route file registers endpoints under `/api`.

3.4 Use Case: AI Intake Form Handling

3.4.1 Purpose

The AI Intake Form allows users to submit structured responses. The server processes and stores these submissions in Azure Blob Storage as a CSV file.

3.4.2 Architectural Role

The server acts as a persistence layer between the frontend and Azure Blob Storage.

- Receives structured JSON payloads
- Converts data into a flat file format (CSV)
- Writes the file to Azure Blob Storage

No database is used. Blob Storage serves as the sole persistence layer for survey data.

3.5 Use Case: Document Retrieval and Editing (WebViewer Integration)

3.5.1 Purpose

The server provides file-based endpoints to support document viewing and editing within the frontend using Apryse WebViewer: WebViewer: JavaScript Document SDK | Apryse.

3.5.2 Architectural Role

The server acts as a secure file gateway between the frontend of the WebViewer and Azure Blob Storage. It exposes

- A GET endpoint to stream documents from Blob Storage
- A POST endpoint to upload updated documents back to Blob

The server does not modify document content, as the document editing one of the WebViewer features. It performs file transfer and storage operations only.

3.6 Relationship with Frontend

The server is consumed by the frontend via relative API paths:

- AI Intake Form submission: `/server/storeResponse`
- WebViewer document retrieval: `/server/docx/...`

- WebViewer document save: `/server/docx/saveEdits `

The server does not render UI or manage sessions. It acts strictly as data storage intermediary and document file gateway.

Section 4: Backend

4.1 Framework/structure

The backend component is a Unvicorn project (which is running FastAPI and Python v3.10) located under `./backend/` with all source code found in `./backend/ai_ml_tools/`. This component handles all API requests from the frontend for AI/ML related models and tools.

Using a terminal shell, the frontend can be started by running the `uvicorn ai_ml_tools.main:app --port 8000 ` command. It is recommended to use a virtual environment which is included in the `.gitingore` and `./backend/.dockerignore` files to prevent unnecessary copying of the virtual environment.

When using Docker, it is deployed as its own container with a volume for a Chroma storage.

All dependencies used in this component are defined in the `requirements.txt` file, and it is important to use Python v3.10 as other python versions will likely fail when installing the libraries.

Multiple VMs (local and on Azure) have been used for hosting the models for Scale Ageing and French Translations. To update which VM the backend should connect too, modify the `http_api` path found in `ai_ml_tools/routers/age_scale.py` and `ai_ml_tools/routers/french_translations.py`.

4.2 Environment Configuration and Azure Key Vault

The backend uses environment variables stored in `backend/ai_ml_tools/.env`. The required environment variables are used to create connections to OpenAI models, Document Intelligence, Azure Machine Learning Workspace, and computer vision models. Some are read on startup while others are dynamically read during API calls.

When running locally, the keys should be included for the AI Hub to function properly; however, they are not needed when deploying to Azure due to the use of Azure Key Vault (AKV). Each key that should be deleted before pushing to Azure is indicated by a comment in the `.env.sample` file stating to leave it blank when pushing to Azure.

The file `ai_ml_tools/core/keyvault.py` establishes the connection to AKV or determines if AKV is not being used. This is dependent if the ` KEY_VAULT_NAME` environment variable is present (which is only found in the Azure Web App not the `.env` file).

The file `ai_ml_tools/core/secrets.py` contains the `get_secret(name)` function to either retrieve the key from the environment variables or from AKV, using a cache to limit the number of external requests made.

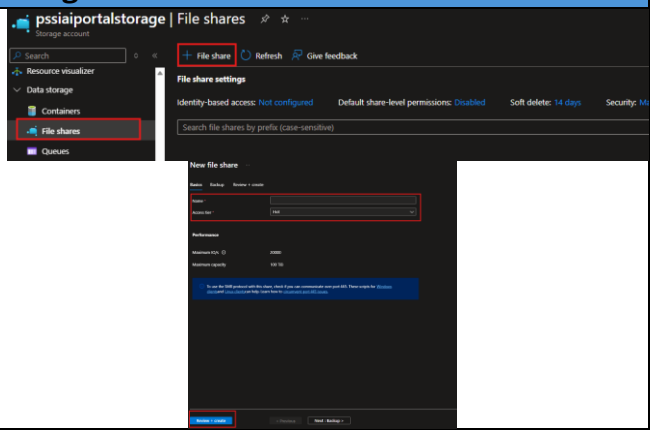
The file `ai_ml_tools/utils/azure_key_vault.py` contains the helper functions that call the `get_secret(name)` function used by any other file needing to retrieve keys in the backend for external API calls.

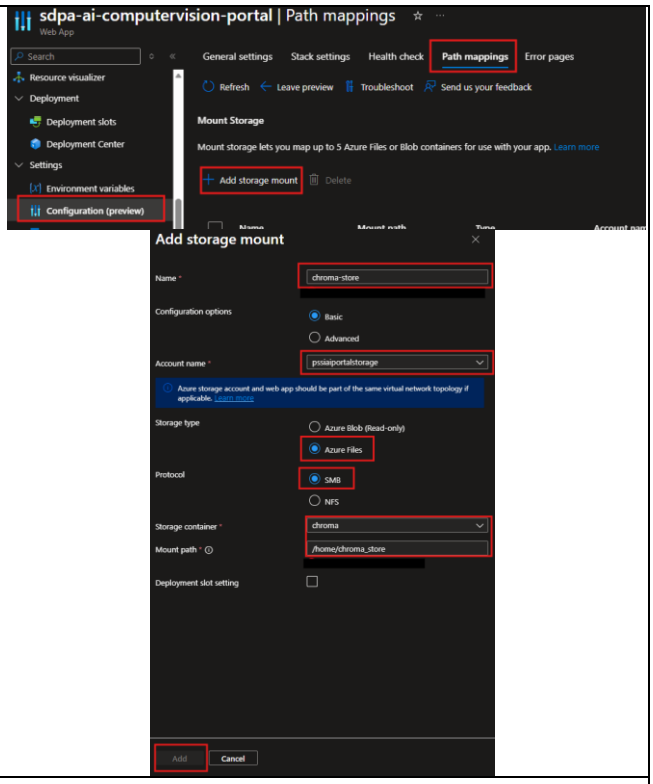
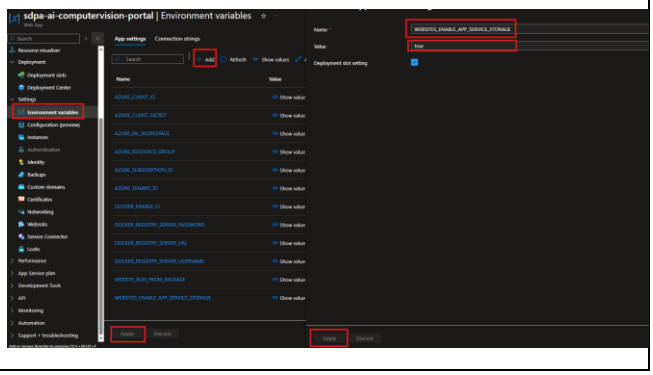
4.3 Choma Storage in Volumes and File Share

The Web Scraper tool uses `langchain_chroma` for storing collected content when scraping websites which can then be filtered by an embedding model before being used to answer a user's question with OpenAI. To ensure a website only needs to be scraped once, persistent storage is needed for the Chroma database.

To allow persistent storage of the Chroma data, the `docker-compose.yml` file creates the volume `chroma_store:/home/chroma_store` in the backend. The web scraper tool then needs to define `/home/chroma_store`as the persistent directory when defining the Chroma database to access the volume.

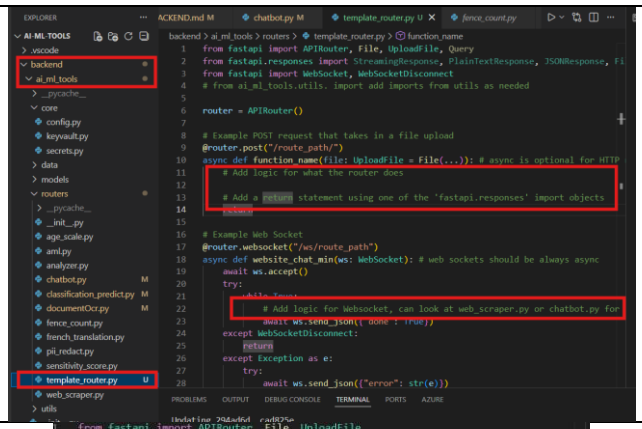
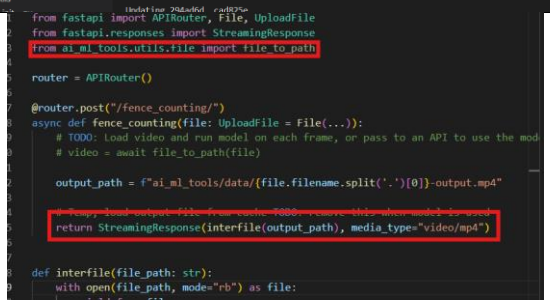
A complication is introduced when deploying the AI Hub on Azure Web Apps. This is because Azure does not support volumes like native docker does. To solve this issue, an Azure Storage Account file share can be used in place of the docker volume. The steps in setting this Azure file share are:

Step Description	***Image***
Create a new file share in an Azure Storage Account (in the File Shares section), call it `chroma`. The recommended access tier is `hot`.	

In the Azure Web App resource, go to path mappings found under the Configuration page and add a new storage mount. 1. Set the name to `chroma-store` 2. Set the account name to the Azure Storage Account used for the file share. 3. Select `Azure Files` as storage type 4. Select `SMB` for protocal 5. Choose your file share for the storage container 6. Set the mount path to `/home/chroma_store` 7. Click on the add button	
In the Azure web app resource, go to environment variables and set `WEBSITES_ENABLE_APP_SERVICE_STORAGE` to `true`. Click on Apply to update the variable, then click Apply on the variables page to save the change.	

4.4 Adding a New API Route

Step Description	***Image***

Go to the `backend/ai_ml_tools/routers` folder. • If you are adding a route for a new tool, copy and rename the `template_router.py` file. • If a file for your tool already exits, open that file.	
Add the logic for the HTTP and/or WS routes as needed. Helper functions can be added to the `ai_ml_tools/utils` folder and class definitions can be added to the `ai_ml_tools/models` folder then imported into the router file.	
After building the new routers, go to the `backend/ai_ml_tools/routers/__init__.py` file. Ensure your router file is imported and included using the `include_router()` function.	

Next time the backend is run, it will now serve the new routes that have been added.

4.5 Supported APIs

4.5.1 HTTP Requests Handled Internally

Requests handled in the `ai_ml_tools/routers/age_scale.py` file:

- HTTP"/age_scale/" - POST request that takes a TIFF image, preprocess it, then calls an external VM with HTTP post request to obtain and return the model predicted age of the fish.
- HTTP"/to_png" - POST request that a TIFF image, convert it to a PNG file, then return it.

Requests handled in the `ai_ml_tools/routers/aml.py` file:

- HTTP”/models” - GET request that retrieves all models in the Azure Machine Learning Workspace.
- HTTP "/models/{name}/versions" - GET request that retrieves all versions of the model based on name provided in `{name}`.
- HTTP "/models/{name}/versions/{version}/download.zip" - GET request that retrieves the files and folders for the model and version specified by `{name}` and `{version}` respectively. The retrieved files are converted into a ZIP file before being returned.
- HTTP "/models/{name}/versions/{version}/readme" - GET request that retrieves the README file if it exists from the model and version specified by `{name}` and `{version}` respectively.

Requests handled in the `ai_ml_tools/routers/analyzer.py` file:

- HTTP"/openai_csv_analyze/" - POST request that takes both a CSV and PDF. Reads the CSV and applies those prompts to the PDF using LLM. Returns the model responses in the from of a JSON, CSV, or TXT depending on what the `outputType` input is set too.

Requests handled in the `ai_ml_tools/routers/chatbot.py` file:

- HTTP"/di_extract_document/" - POST request that takes a PDF document then uses an Azure document intelligence prebuilt model to convert the PDF into a stringified JSON and return it.
- WS"/ws/chat_stream" - Web socket that will create chunked objects with documents string, get relevant chunks to the given question using an embedding model, then ask the question on the selected document chunks with a LLM, the response is returned as a stream (in chunks).
- HTTP"/di_chunk_single_document/" - POST request that takes a single PDF document and uses an Azure document intelligence prebuilt model to convert a PDF into markdown chunks, they are combined into a JSON containing text chunks and metadata then returned.
- HTTP"/di_chunk_ multi_document/" - POST request that takes multiple PDF documents and uses an Azure document intelligence prebuilt model to convert a PDF into markdown chunks, they are combined into a JSON containing text chunks and metadata then returned.

Requests handled in the `ai_ml_tools/routers/classification_predict.py` file:

This tool is currently under development, and requests are activity changing in this file. Documentation will be added once this tool is complete.

Requests handled in the `ai_ml_tools/routers/documentOcr.py` file:

- HTTP”/index” - POST request that takes a set of PDF files, extracts text from multiple documents using Azure document intelligence, creates a vector store, then returns the vector store.
- HTTP” /extract_per_file” - POST request that takes the vector store ID, fields, document names, and model type. It will apply a LLM to extract the specified fields from the documents and return for each document the name, fields, LLM answers, sources, and reasonings.

Requests handled in the `ai_ml_tools/routers/fence_count.py` file:

- HTTP"/fence_counting/" - POST request that takes a file name and returns a cached output video based on the inputted file name.

Requests handled in the `ai_ml_tools/routers/french_translation.py` file:

- HTTP"/pdf_to_french/" - POST request that takes in a PDF, extracts the raw text, then redirects to "/text_to_french/" HTTP request.
- HTTP"/text_to_french/" - POST request that takes in raw text then calls external VM with HTTP request to convert the text to French, the text response from the VM is returned.

Requests handled in the `ai_ml_tools/routers/pii_redact.py` file:

- HTTP"/pii_redact/" - POST request that takes in a PDF, uses the `fitz` library to extract the text, determines sensitive information using `presidio`, redacts sensitive information, then returns the redacted PDF.

Requests handled in the `ai_ml_tools/routers/sensitivity_score.py` file:

- HTTP"/sensitivity_score/" - POST request that takes in a PDF, uses the `fitz` library to extract the text, determines all sensitive information by type using `presidio`, then returns a calculated sensitivity score.

Requests handled in the `ai_ml_tools/routers/web_scraper.py` file:

- HTTP "/scrape" - POST request that takes a URL and scrapes a it (or uses cached data), stores chunks in memory, upserts them to Chroma, and returns a session_id.

- HTTP "/scrape/{session_id}/combined.txt" - GET request that retrieves and returns the raw combined scraped text for a fresh scrape as a downloadable TXT file. Scraped text is from the URL scape tied to the provided `{session_id}`.
- HTTP "/presets" - GET request that returns a list of cached URLs (presets) discovered in the vector store.
- HTTP "/combined-by-url" - GET request that combines all the chunks from a single URL into a string which is then returned in a TXT file.
- HTTP "/base-presets" - GET request that retrieves and returns a list of the unique base domains from vector DB.
- HTTP "/pages" - GET request that retrieves and returns all pages (full URLs) belonging to a given base domain.
- WS "/ws/website_chat" - Web socket that takes a user's question, retrieves relevant chunks from the vector store for the given URL, then applied LLM to generate a response that gets returned.

4.5.2 External APIs Used

Azure OpenAI

Azure Open AI is used for large language models for generating AI responses and embedding models for implementing RAG. The PDF Chatbot, CSV/PDF Analyzer, Web Scraper, and Document OCR tools rely on OpenAI models.

Azure Open AI provides advanced machine learning models that can interpret and respond to user queries, and analyze text data extracted from documents. This integration allows for:

- Enhanced natural language processing for real-time conversation with documents.
- Sophisticated data extraction and analysis from various file formats, enabling deep insights into document content.

Azure Document Intelligence

Document Intelligence is used to read and interpret the contents of documents uploaded to the AI Hub's PDF Chatbot, CSV/PDF Analyzer, French Translation, and Document OCR tools. It helps in:

- Automatically extract text and data from structured and unstructured documents including non-machine-readable files.

- Organize extracted text into markdown format allowing for LLMs and embedding models to easily process document text.

Azure Machine Learning Workspace

The Azure Machine Learning workspace is a central hub for managing all the artifacts and resources needed for machine learning projects. It provides a collaborative environment where teams can create, manage, and deploy machine learning models. It is used by the Models tool and helps in:

- Acting as a cloud storage for AI/ML models that users can upload, explore, and manage through the AI Hub interface.

Azure Custom Vision

Custom Vision is used for custom built computer vision models, specifically for the Classification Model tool. Custom Vision allows for API calls directly to its existing models, this allows for the AI Hub too:

- Have access to computer vision classification models that have been trained in the Custom Vision resource.
- Process images uploaded on the AI Hub and use the computer vision model to provide a prediction with a confidence score.

External VMs

External VMs allow for tools to use more computationally heavy models without overburdening the backend component of the AI Hub. The Scale Ageing and French Translation tools use external VMs for hosting their models. The backend component manages the API connection to these VMs.